Avaliação Experimental de Configurações na Identificação Interativa de Fronteiras de Microsserviços com a Ferramenta i-toMicroservices

Hélio Toshio Kamakawa
Universidade Estadual de Maringá (UEM)
pg56583@uem.br

Dacio Fernando Machado Francisco
Universidade Estadual de Maringá (UEM)
pg406906@uem.br

João Vitor Staub Castanho
Universidade Estadual de Maringá (UEM)
pg406025@uem.br

João Choma Neto
Universidade Estadual de Maringá (UEM)
jcneto2@uem.br

Thelma Elita Colanzi
Universidade Estadual de Maringá (UEM)
teclopes@uem.br

***Resumo*—A identificação de fronteiras de microsserviços em sistemas legados exige equilibrar critérios conflitantes e incorporar o conhecimento de domínio dos arquitetos. A ferramenta i-toMicroservices aborda esse problema por meio de otimização interativa, em que um tomador de decisão (DM) humano intervém durante a busca. Este trabalho avalia três mecanismos recentemente incorporados à ferramenta para preservar a intenção do DM: o elitismo das soluções modificadas por ele, o tratamento da restrição de balanceamento e o algoritmo R-NSGA-III. Oito configurações foram executadas cinco vezes em dois sistemas, o spring-petclinic e o jpetstore, totalizando 80 execuções, com o DM guiado por uma persona que prioriza fronteiras baseadas no domínio. As fronteiras finais foram comparadas pelos indicadores HV, IGD+ e ED, e, no spring-petclinic, as soluções foram confrontadas com uma decomposição de referência definida antes das execuções. O tratamento de restrição foi o fator de maior impacto nos indicadores, com vantagem para a penalização das soluções inviáveis, enquanto o algoritmo e o elitismo não produziram diferenças significativas. No spring-petclinic, a aderência à decomposição de referência aumentou ao longo das intervenções do DM em todas as configurações, mas as soluções totalmente aderentes ocorreram apenas com o descarte das soluções inviáveis, justamente as de pior desempenho nos indicadores. Esses resultados indicam que indicadores de qualidade e preferências do DM podem divergir, e apontam oportunidades de evolução da ferramenta.**

***Palavras-chave*—microsserviços, engenharia de software baseada em busca, otimização interativa, preferências do tomador de decisão, algoritmos genéticos**

I. Introdução

A migração de sistemas monolíticos para arquiteturas de microsserviços tem sido adotada por organizações que buscam maior escalabilidade, autonomia das equipes e facilidade de manutenção. Em um monólito, todas as funcionalidades compartilham uma única base de código e uma única unidade de implantação; à medida que o sistema cresce, alterações localizadas passam a exigir a reimplantação de todo o sistema, e escalar uma parte dele implica replicar o conjunto inteiro. A arquitetura de microsserviços contorna essas limitações ao decompor o sistema em serviços pequenos e autônomos, implantados de forma independente. Nesse processo, uma das etapas mais críticas é a identificação das fronteiras dos microsserviços, isto é, a decisão de quais elementos do código legado devem compor cada serviço. Apesar do grande número de abordagens propostas, a decomposição de monólitos em microsserviços ainda é uma área em amadurecimento, com pouco suporte ferramental e critérios de avaliação heterogêneos [1].

Tradicionalmente, essa identificação é feita manualmente, com base na análise do código-fonte e na experiência subjetiva dos arquitetos, o que exige tempo e esforço consideráveis e tende a não escalar à medida que o sistema cresce [2], [3]. Para apoiar essa tarefa, foram propostas abordagens automatizadas, entre elas as baseadas em busca, que tratam a decomposição como um problema de otimização. A identificação de microsserviços exige equilibrar simultaneamente critérios conflitantes, como coesão, acoplamento, *overhead* de comunicação, modularização de funcionalidades e reúso. A abordagem toMicroservices trata esse problema como uma otimização com muitos objetivos, resolvida pelo algoritmo NSGA-III [4], [5].

Abordagens totalmente automatizadas, porém, operam isoladas do conhecimento tácito de domínio dos arquitetos, como "caixas-pretas" que não permitem refinar as soluções durante a busca, e podem produzir partições desbalanceadas, que exigem ajustes antes de serem adotadas [3], [6]. Um estudo industrial mostrou, ainda, que mantenedores com perfis distintos têm preferências divergentes sobre a granularidade e a composição dos microsserviços, e apontou a necessidade de abordagens interativas, que incorporem essas preferências durante a busca [7]. Para atender a essa demanda, foi desenvolvida a ferramenta i-toMicroservices, extensão interativa da abordagem, que permite que um tomador de decisão (*Decision Maker*, DM) humano interrompa a evolução e modifique as soluções candidatas [6]. Um estudo exploratório indicou que essa intervenção humana pode corrigir desbalanceamentos das partições geradas automaticamente e isolar subdomínios de negócio que a otimização sozinha não separa [3].

Recentemente, três mecanismos foram incorporados à

i-toMicroservices com o propósito de preservar a intenção do DM ao longo da evolução: o elitismo das soluções modificadas pelo DM, o tratamento da restrição de balanceamento (PENALTY ou KILL) e o algoritmo R-NSGA-III, concebido para concentrar a busca nas regiões de interesse do tomador de decisão [10]. Até onde se sabe, esses mecanismos ainda não haviam sido avaliados em uso interativo, com um DM humano. Além disso, eles podem gerar efeitos opostos: um mecanismo que melhora a qualidade da fronteira segundo os indicadores tradicionais não necessariamente produz soluções mais próximas da decomposição desejada pelo DM, e vice-versa. O objetivo deste trabalho é avaliar se esses mecanismos são benéficos ou prejudiciais no uso interativo da ferramenta, considerando separadamente essas duas perspectivas. Para isso, são investigadas duas questões de pesquisa:

- **RQ1**: Qual configuração teve melhor desempenho em termos de indicadores de qualidade?
- **RQ2**: Os resultados retornados pela ferramenta i-toMicroservices estão condizentes com as preferências da persona que interagiu durante o processo de otimização?

Para responder a essas questões, foi conduzido um experimento com oito configurações, resultantes da combinação dos três mecanismos, aplicadas a dois sistemas de código aberto (spring-petclinic e jpetstore). Para controlar seu comportamento, o DM humano interagiu com a ferramenta segundo uma persona predefinida, que estabelece suas prioridades, as operações permitidas e o momento das decisões. Cada configuração foi executada cinco vezes por sistema, totalizando 80 execuções. As fronteiras finais foram comparadas pelos indicadores Hypervolume, IGD+ e distância euclidiana ao ideal, com testes estatísticos não paramétricos e medida de tamanho de efeito, e, no spring-petclinic, as soluções foram confrontadas com a decomposição esperada pela persona. Os resultados indicam que o tratamento de restrição é o fator de maior impacto nos indicadores de qualidade, com vantagem para PENALTY, mas que, no spring-petclinic, as soluções mais aderentes às preferências da persona ocorreram com KILL, o que sugere que indicadores e preferência do DM podem apontar em direções opostas.

As principais contribuições deste trabalho são uma avaliação experimental de oito configurações da i-toMicroservices em dois sistemas, com um DM humano no ciclo de otimização; uma análise da aderência das soluções retornadas às preferências de domínio da persona; e um conjunto de sugestões de melhoria para a ferramenta, derivadas dos resultados e da experiência de uso.

O restante do artigo está organizado da seguinte forma: a Seção II apresenta a fundamentação teórica; a Seção III descreve a metodologia; a Seção IV apresenta e discute os resultados; a Seção V reúne sugestões de melhoria para a ferramenta; e a Seção VI traz as considerações finais.

II. Fundamentação Teórica

A. Otimização em Engenharia de Software

A Engenharia de Software Baseada em Busca (*Search-Based Software Engineering*, SBSE) modela problemas de engenharia de software como problemas de otimização, resolvidos por metaheurísticas como os algoritmos evolutivos [8]. Nessa formulação, uma solução candidata (por exemplo, uma decomposição do sistema em microsserviços) é codificada em uma representação manipulável pelo algoritmo, e sua qualidade é medida por funções objetivo. Em muitos problemas de engenharia de software, os objetivos são conflitantes: melhorar um deles tende a piorar outro.

Um problema de otimização multiobjetivo pode ser definido como

$$\min_{x \in \Omega} F(x) = \big(f_1(x), f_2(x), \ldots, f_m(x)\big), \tag{1}$$

em que Ω é o espaço de soluções e $f_1, \ldots, f_m$ são as m funções objetivo. Objetivos de maximização podem ser convertidos em minimização, por exemplo, pela inversão de seus valores. Uma solução x *domina* uma solução y quando $f_i(x) \leq f_i(y)$ para todo $i \in \{1, \ldots, m\}$ e $f_j(x) < f_j(y)$ para pelo menos um j. O conjunto das soluções não dominadas forma a fronteira de Pareto, que representa os melhores compromissos disponíveis entre os objetivos [8]. Quando $m > 3$, fala-se em otimização com muitos objetivos (*many-objective*): nesse cenário, a proporção de soluções não dominadas cresce rapidamente, a pressão seletiva baseada apenas em dominância se enfraquece e a fronteira se torna difícil de visualizar e de interpretar pelo tomador de decisão.

Como o algoritmo retorna um conjunto de soluções, e não uma única resposta, é necessário incorporar as preferências de um tomador de decisão para escolher entre elas. Essas preferências podem ser fornecidas antes da busca (*a priori*), apenas ao final, sobre a fronteira obtida (*a posteriori*), ou durante a própria busca, de forma *interativa*. Na otimização interativa, o tomador de decisão avalia e modifica soluções intermediárias, e suas escolhas orientam as gerações seguintes; com isso, o algoritmo concentra o esforço nas regiões de interesse, e o tomador de decisão não precisa escolher, ao final, entre um número muito grande de soluções. Em contrapartida, cada interação tem custo cognitivo, e interações excessivas geram fadiga [3].

O NSGA-III é um algoritmo genético projetado para problemas com muitos objetivos [9]. A cada geração, a população de pais é combinada à de descendentes, e o conjunto resultante é ordenado em frentes de não dominância. Para escolher, entre as soluções da última frente admitida, quais seguem para a próxima geração, o algoritmo normaliza os objetivos e associa cada solução ao ponto mais próximo de um conjunto de pontos de referência distribuídos uniformemente sobre um hiperplano, priorizando os pontos menos ocupados. Os pontos de referência são gerados pela divisão de cada eixo em p partes, o que produz $\binom{m+p-1}{p}$ pontos; com $m = 5$ objetivos e $p = 5$ divisões, por exemplo, são 126 pontos. Esse mecanismo mantém a diversidade da população ao longo de toda a fronteira [9].

O R-NSGA-III estende o NSGA-III para concentrar a busca nas regiões de interesse do tomador de decisão [10]. Em vez de distribuir os pontos de referência por todo o hiperplano, ele os gera em torno de pontos de aspiração fornecidos pelo tomador de decisão, em uma vizinhança cuja extensão é controlada por um fator de contração. Assim, a população converge para a porção da fronteira de Pareto mais próxima das aspirações, em vez de cobri-la por inteiro. Essa característica é especialmente relevante na otimização interativa, em que as aspirações podem ser atualizadas a cada intervenção do tomador de decisão.

Dois outros conceitos são relevantes para este trabalho. O *elitismo* consiste em preservar as melhores soluções de uma geração para a seguinte, evitando que sejam perdidas pelos operadores genéticos; o NSGA-III já é elitista, pois seleciona os sobreviventes entre pais e descendentes. Em um contexto interativo, o elitismo pode ser estendido às soluções modificadas pelo tomador de decisão, que passam a ser reinjetadas na população para que a intervenção não se perca. O *tratamento de restrições*, por sua vez, define o destino das soluções que violam alguma restrição do problema. Duas estratégias clássicas são a penalidade, que mantém a solução na população, mas piora seus valores de objetivo na proporção da violação, e a pena de morte, que descarta a solução inviável. A primeira preserva a diversidade e permite atravessar regiões inviáveis do espaço de busca; a segunda garante que apenas soluções viáveis sobrevivam, ao custo de reduzir a exploração.

B. Modernização de Sistemas Legados com Microsserviços

Microsserviços são serviços pequenos e autônomos, cada um responsável por uma capacidade de negócio, que se comunicam por mecanismos leves e podem ser implantados de forma independente. A migração de um monólito para essa arquitetura envolve, em linhas gerais, coletar dados sobre o sistema legado, analisá-los para identificar os candidatos a microsserviços e avaliar a decomposição resultante. Os dados de entrada podem ser estáticos, como as dependências extraídas do código; dinâmicos, como os rastros de execução; ou evolutivos, como o histórico de alterações do repositório. Uma revisão sistemática de 35 estudos concluiu que a área ainda está em estágio inicial, que faltam métodos que combinem esses três tipos de dados e que o suporte ferramental é limitado [1]. Além disso, as técnicas automatizadas costumam ser avaliadas em sistemas e critérios distintos, o que dificulta compará-las e medir o progresso da área [2].

A abordagem toMicroservices identifica candidatos a microsserviços por meio de uma busca com muitos objetivos [4]. O sistema legado é modelado como um grafo de dependências no nível de métodos, cujas arestas combinam chamadas estáticas, rastros dinâmicos de execução e o tamanho estimado dos dados trafegados [3]. Cada solução representa uma alocação desses métodos a um conjunto de microsserviços e é avaliada por cinco critérios indicados por praticantes [4], [5]: (i) **acoplamento**, o grau de dependência entre os microsserviços (a minimizar); (ii) **coesão**, o quanto os métodos de um mesmo microsserviço se relacionam entre si (a maximizar); (iii) ***overhead* de rede**, o volume de dados trafegados entre microsserviços (a minimizar); (iv) **modularização de funcionalidades**, o quanto cada microsserviço concentra uma única funcionalidade (a maximizar); e (v) **reúso**, o potencial de reaproveitamento dos microsserviços (a maximizar). A abordagem foi avaliada em um sistema legado industrial, e a maioria dos candidatos recomendados foi considerada adotável pelos desenvolvedores [4]. Uma análise posterior, baseada em indicadores de desempenho como o HV e o IGD, mostrou que esses critérios são conflitantes entre si e que o NSGA-III supera a busca aleatória na identificação de microsserviços [11].

Como as preferências sobre a granularidade e a composição dos microsserviços variam entre mantenedores [5], [7], não basta que o DM intervenha: sua intenção precisa persistir ao longo da evolução. A ferramenta i-toMicroservices insere o DM no ciclo evolutivo [6]. A busca é executada de forma autônoma por um número predefinido de gerações e, em pontos de interação, é pausada para que o DM inspecione as soluções candidatas, escolha uma delas e a modifique por meio de quatro operações: *mover* métodos entre microsserviços, *dividir* um microsserviço em dois, *unir* dois microsserviços e *congelar* um microsserviço, impedindo que sua composição seja alterada nas gerações seguintes. Como interações frequentes geram fadiga, as modificações do DM são propagadas automaticamente para soluções semelhantes da população: após cada interação, a população é dividida em grupos de soluções com perfis de objetivos similares, e a alteração é replicada nas soluções do grupo da solução modificada, de modo que uma única intervenção influencie várias soluções [3], [6]. Após a interação, a evolução é retomada.

Em um estudo exploratório com três engenheiros de software, a intervenção humana aumentou em até 23,7% a similaridade das decomposições com arquiteturas de referência e permitiu isolar subdomínios de negócio coesos; os resultados também sugeriram que a qualidade da decomposição depende do perfil do DM [3]. Essa dependência motiva o uso de uma persona com perfil definido no experimento descrito a seguir. Os mecanismos avaliados neste trabalho (elitismo, tratamento de restrição e R-NSGA-III) têm o mesmo propósito da propagação, o de preservar a intenção do DM, e sua configuração é descrita na Seção III-B.

III. Metodologia

O estudo foi conduzido como um experimento controlado em que um DM humano interage com a ferramenta i-toMicroservices durante o processo evolutivo. Foram avaliadas oito configurações algorítmicas em dois sistemas, com cinco execuções independentes por configuração, totalizando 80 execuções.

A. Sistemas Utilizados

Foram utilizados dois sistemas monolíticos de código aberto, amplamente adotados como *benchmarks* na avaliação de técnicas de decomposição em microsserviços [2]. O **spring-petclinic** é um sistema de gestão de clínica veterinária, com 23 classes e 90 métodos [2]; a ferramenta opera sobre 67 métodos

de 14 classes de negócio, distribuídos em quatro domínios (Tabela II). O **jpetstore** é uma loja virtual de animais de estimação, com 24 classes e 298 métodos [2], cujos domínios de negócio considerados incluem *catalog*, *cart* e *order*.

Cada sistema foi executado por um participante distinto: o spring-petclinic pelo primeiro autor e o jpetstore pelo segundo autor, ambos assumindo a mesma persona (Seção III-C).

B. Configurações dos Algoritmos

As configurações resultam da combinação de três fatores, cada um com dois níveis: (i) **algoritmo**: NSGA-III, que explora a fronteira de forma ampla [9], ou R-NSGA-III, que concentra a busca em torno de pontos de aspiração [10], presumivelmente derivados da solução escolhida pelo DM; (ii) **elitismo**: quando ativo, espera-se que as soluções modificadas pelo DM sejam preservadas nas gerações seguintes; quando inativo, essa preservação não é garantida; e (iii) **tratamento de restrição**: PENALTY penaliza as decomposições desbalanceadas e as mantém na busca, enquanto KILL as descarta. A Tabela I apresenta as oito configurações na ordem em que foram executadas. Os pares consecutivos (C1–C2, C3–C4, C5–C6 e C7–C8) diferem apenas no elitismo.

Tabela I
CONFIGURAÇÕES AVALIADAS

Config.	Algoritmo	Elitismo	Restrição
C1	NSGA-III	ativo	PENALTY
C2	NSGA-III	inativo	PENALTY
C3	R-NSGA-III	ativo	PENALTY
C4	R-NSGA-III	inativo	PENALTY
C5	NSGA-III	ativo	KILL
C6	NSGA-III	inativo	KILL
C7	R-NSGA-III	ativo	KILL
C8	R-NSGA-III	inativo	KILL

Os demais parâmetros foram mantidos fixos em todas as execuções: população de 132 indivíduos, 1.000 gerações, quatro microsserviços iniciais e cinco divisões para a geração dos pontos de referência. Para a restrição de balanceamento, foram adotadas as proporções mínima e máxima de 0,02 e 0,3 por microsserviço e, nas configurações PENALTY, fator de penalidade 10. Cada configuração foi executada cinco vezes por sistema. Em cada execução, a ferramenta pausou a evolução em três pontos de interação, nas gerações 600, 820 e 950 (60%, 82% e 95% da execução), para que o DM escolhesse e modificasse uma solução. Em todas as execuções foram otimizados os cinco objetivos descritos na Seção II-B.

As execuções do spring-petclinic foram realizadas em uma máquina virtual VirtualBox com Ubuntu 24.10 (64 bits), dois processadores e 5 GB de memória, com a ferramenta executada em contêineres Docker.

C. Descrição da Persona

O DM atuou segundo a persona P1, *Arquiteto Reflexivo*, que concebe a decomposição como um exercício de modelagem de domínio: cada microsserviço deve corresponder a um único conceito de negócio. Para essa persona, a coesão semântica de domínio é o objetivo primário; a piora do acoplamento e do *overhead* de rede é aceita como custo de transição, e a funcionalidade e o reúso são vistos como consequência. Quanto ao comportamento, a persona sempre escolhe o primeiro candidato exibido, utiliza apenas a operação *Mover* (sem dividir, unir ou congelar microsserviços) e concentra suas decisões no fim do processo evolutivo.

Para reduzir o viés do DM, a decomposição desejada pela persona foi definida antes das execuções, na forma de um gabarito que associa cada um dos 67 métodos do spring-petclinic a um dos quatro domínios (Tabela II). A associação foi feita no nível de classe: cada entidade e seu respectivo controlador formam um domínio, e as superclasses compartilhadas (`BaseEntity` e `NamedEntity`) foram atribuídas ao domínio *owner*. As decisões do DM em todas as execuções do spring-petclinic foram orientadas por esse gabarito, e a **aderência** de uma solução é o percentual dos 67 métodos alocados no microsserviço correspondente ao seu domínio, considerando a associação entre microsserviços e domínios que maximiza esse percentual. O gabarito completo, com o pacote, a classe e o domínio de cada um dos 67 métodos, está disponível no material complementar, na pasta `dataset` (Seção III-F).

Tabela II
GABARITO DE DOMÍNIOS DO SPRING-PETCLINIC

Domínio	Classes	Métodos
owner	Owner, OwnerController, Person, BaseEntity, NamedEntity	32
pet	Pet, PetController, PetValidator, PetTypeFormatter	18
vet	Vet, VetController, Vets	9
visit	Visit, VisitController	8
Total	14 classes	67

Para tornar as decisões reproduzíveis, foi adotado no spring-petclinic um protocolo fixo. Em cada interação, o DM realiza dez movimentos de métodos, na ordem de prioridade *vet*, *visit*, *pet* e *owner*. Um microsserviço é considerado "arrumado" quando contém ao menos 80% dos métodos de seu domínio e quando ao menos 80% de seus métodos pertencem a esse domínio; microsserviços arrumados são pulados, e a interação é encerrada antes dos dez movimentos quando todos estão arrumados. Esse protocolo foi adotado a partir da C2: nas cinco execuções da C1, conduzidas antes de sua definição, a quantidade de movimentos por interação não estava fixada e variou de 0 a 17.

D. Indicadores de Qualidade e Testes Estatísticos

A análise considera a fronteira não dominada da última geração de cada execução. Como os objetivos têm escalas distintas, os valores foram normalizados para $[0, 1]$ com os limites mínimo e máximo obtidos do conjunto de todas as fronteiras do mesmo sistema. Todos os objetivos foram tratados como de minimização: coesão, modularização de funcionalidades e reúso foram armazenados na forma inversa ($1/x$). A fronteira de referência de cada sistema é o conjunto não dominado da união de todas as suas fronteiras normalizadas.

Sobre esse espaço comum foram calculados três indicadores [8]: (i) **Hypervolume** (HV), com ponto de referência $1{,}1$ em cada objetivo (maior é melhor); (ii) **IGD+**, a distância da fronteira de referência até a fronteira obtida (menor é melhor); e (iii) **ED**, a distância euclidiana média das soluções da fronteira ao ponto ideal $(0, 0, 0, 0, 0)$ (menor é melhor). No spring-petclinic, o reúso foi constante em todas as soluções, foi normalizado como zero e, portanto, não contribui para diferenciá-las nos indicadores. Os indicadores são calculados por execução, sobre a fronteira final de cada uma, de modo que cada configuração reúne cinco valores de cada indicador; esses cinco valores são resumidos pela mediana nas tabelas e usados integralmente nos testes estatísticos. Cabe destacar que os três indicadores avaliam a qualidade da fronteira segundo os objetivos da ferramenta, e não a proximidade das soluções em relação à decomposição desejada pela persona, avaliada separadamente pela aderência ao gabarito (Seção III-C).

As configurações foram comparadas separadamente por sistema e por indicador, com $n = 5$ execuções por grupo. A normalidade foi verificada pelo teste de Shapiro-Wilk e a homogeneidade das variâncias pelo teste de Levene. Como, com $n = 5$, esses testes têm baixo poder e os pressupostos não foram atendidos em todos os casos, adotou-se de forma uniforme o teste não paramétrico de Kruskal-Wallis, seguido do teste de Dunn quando significativo. Os p-valores das 28 comparações par a par foram ajustados pelo procedimento de Benjamini-Hochberg (FDR), ao nível de significância de 0,05. A magnitude das diferenças foi estimada pelo $\hat{A}_{12}$ de Vargha-Delaney [12], calculado para todos os pares, independentemente da significância.

E. Ameaças à Validade

Validade interna. A ordem das configurações foi fixa, de modo que o aprendizado do DM ao longo do experimento pode favorecer as últimas configurações. No spring-petclinic, a C1 foi executada antes da adoção do protocolo de dez movimentos por interação, de modo que a intervenção do DM nessa configuração não foi padronizada como nas demais, e, em duas execuções da C2, parte dos movimentos do DM não foi aplicada, o que afeta a comparação da aderência dessa configuração. Ainda nesse sistema, como as configurações KILL foram as últimas executadas, a experiência acumulada pelo DM pode ter contribuído para que as soluções 100% aderentes ocorressem apenas nelas. Por fim, não foram realizadas execuções sem interação do DM, de modo que o aumento da aderência ao longo das execuções não pode ser atribuído exclusivamente às intervenções.

Validade de construto. Os indicadores HV, IGD+ e ED medem a qualidade da fronteira no espaço dos objetivos, e não a aderência semântica ao domínio desejada pela persona; por isso, a RQ2 é avaliada separadamente, pela aderência ao gabarito. O gabarito do spring-petclinic foi elaborado pelo próprio participante e representa uma entre várias decomposições de domínio possíveis; sua definição prévia, contudo, impede que seja ajustado aos resultados da ferramenta.

Validade externa. Foram usados apenas dois sistemas de pequeno porte e uma única persona, o que limita a generalização dos resultados.

Validade de conclusão. Com $n = 5$ por configuração, os testes têm baixo poder estatístico, de modo que diferenças menores, como as eventualmente associadas ao algoritmo e ao elitismo, podem não ter sido detectadas após a correção para múltiplas comparações. Para mitigar essa limitação, o tamanho de efeito é reportado para todos os pares. Além disso, cada sistema contou com um único DM, cuja interpretação da persona pode diferir da de outros participantes.

F. Disponibilidade dos Dados

Para permitir a verificação e a replicação do estudo, o material complementar está disponível publicamente[1]. O material inclui: (i) os pacotes exportados pela ferramenta em cada execução, com as fronteiras finais, as soluções candidatas e o registro das interações do DM; (ii) as tabelas com os valores de HV, IGD+ e ED por execução e com a aderência das soluções ao gabarito; (iii) o gabarito completo, na pasta `dataset`, com a associação de cada um dos 67 métodos à sua classe e ao seu domínio; (iv) os scripts utilizados na análise estatística, no cálculo da aderência e na geração dos gráficos; (v) as figuras complementares, como os gráficos dos indicadores IGD+ e ED; e (vi) a análise descritiva dos dois sistemas, na pasta `dataset/analise_descritiva`, com a estatística descritiva por configuração, os boxplots, os histogramas e gráficos Q-Q, as tabelas de todos os testes estatísticos (Shapiro-Wilk, Levene, Kruskal-Wallis, Dunn com correção de Benjamini-Hochberg e $\hat{A}_{12}$, por par e por fator) e um *notebook* para reproduzi-los no Google Colab.

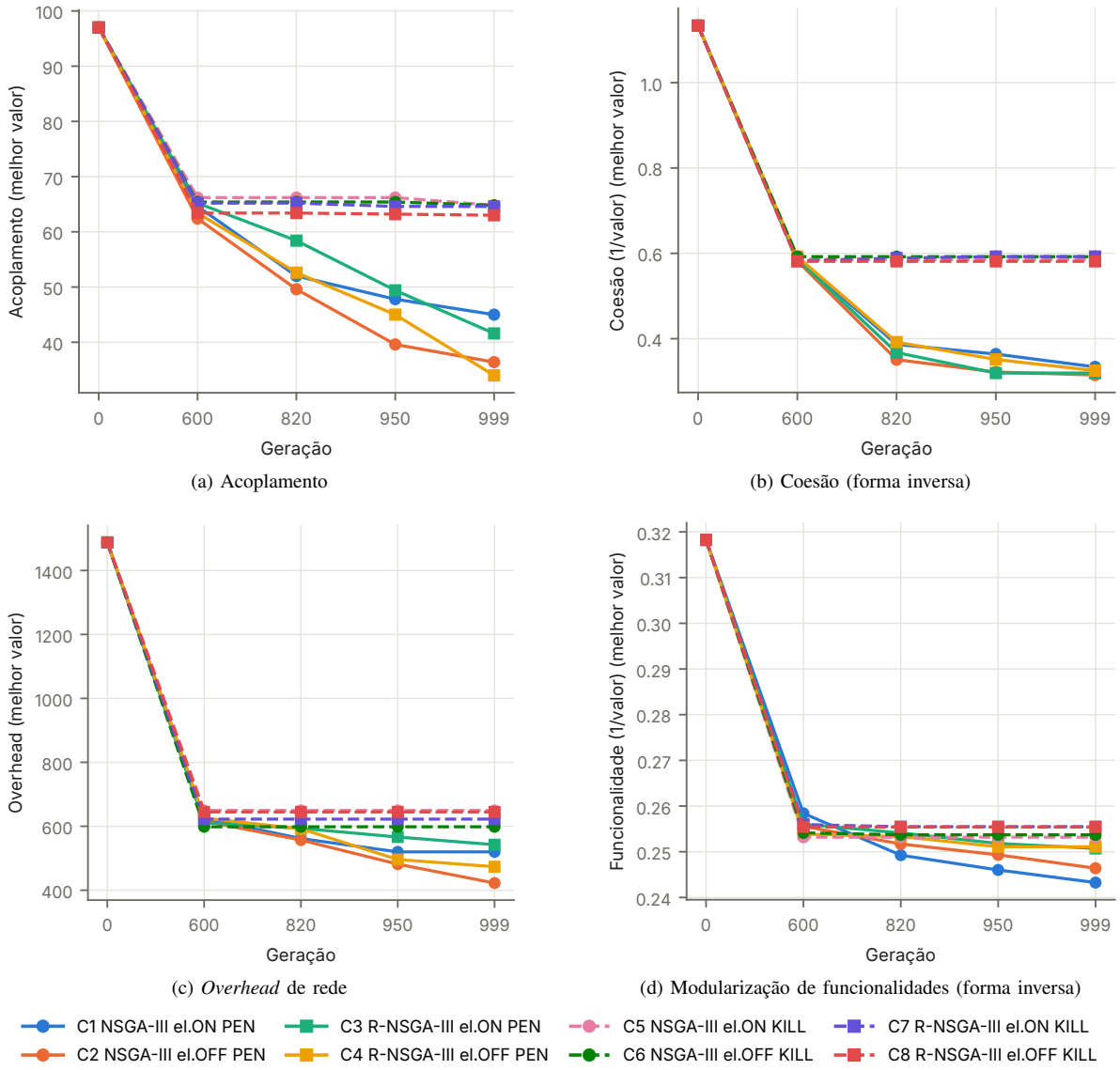
IV. Resultados e Discussão

Todas as 80 execuções foram concluídas até a geração final. No spring-petclinic, foram registradas ocorrências pontuais: em uma execução da C1, o registro das interações da primeira pausa não foi exportado; em duas execuções da C2, movimentos do DM deixaram de ser aplicados em parte das interações; e a máquina virtual foi encerrada logo após o término da execução C5-E5, cujo pacote de dados, no entanto, foi preservado. Como as fronteiras finais de todas as execuções estão completas, nenhuma foi descartada da análise.

A. Convergência do Fitness

A Fig. 1 mostra a evolução do melhor valor de cada objetivo na população do spring-petclinic, da geração inicial à final, passando pelos pontos de interação; cada curva é a média das cinco execuções da configuração, e valores menores são melhores. Todas as configurações apresentam o mesmo melhor acoplamento na geração inicial e evolução praticamente idêntica até a primeira interação: o melhor acoplamento cai de 97 para valores entre 62 e 66 na geração 600. A partir desse ponto, as configurações se separam em dois grupos bem definidos. Com PENALTY (C1 a C4), a busca continua

[1] https://drive.google.com/drive/folders/19EEogYbDqJuWw1V6TR_wJe8y90U67llA



(a) Acoplamento

(b) Coesão (forma inversa)

(c) *Overhead* de rede

(d) Modularização de funcionalidades (forma inversa)

Figura 1. Convergência do fitness no spring-petclinic

evoluindo após as intervenções do DM: na geração final, o melhor acoplamento médio chega a 36,4 na C2 e a 34,0 na C4, e a coesão (na forma inversa) passa de cerca de 0,58 para 0,32. Com KILL (C5 a C8), por outro lado, a população estagna logo após a primeira interação: o melhor acoplamento permanece entre 63 e 65 e a coesão em torno de 0,58 até a geração final. Esse patamar de acoplamento é próximo ao da solução do gabarito (63). Ou seja, segundo os objetivos definidos na modelagem, com KILL os melhores valores deixam de melhorar a partir da primeira interação do DM. O mesmo padrão ocorre com o *overhead* de rede (Fig. 1c) e, de forma menos acentuada, com a modularização de funcionalidades (Fig. 1d). O reúso foi omitido, pois permaneceu constante em todas as soluções do spring-petclinic.

B. Fronteiras Finais

A Fig. 2 apresenta, em coordenadas paralelas, as fronteiras finais das oito configurações. Os valores estão normalizados (0 é o melhor), cada cor corresponde a uma execução, a fronteira de referência aparece em cinza e a linha pontilhada representa a solução do gabarito. A figura permite ler três referências distintas: o valor 0 de cada eixo corresponde ao melhor valor encontrado naquele objetivo, isto é, ao que a modelagem do problema considera ideal, ainda que nenhuma solução o atinja em todos os objetivos simultaneamente; a fronteira de referência, em cinza, reúne, no conjunto das execuções, as melhores soluções alcançadas, isto é, aquelas não superadas por nenhuma outra em todos os objetivos; e a linha pontilhada representa a decomposição definida pelo DM. A distância entre a linha pontilhada e a fronteira de

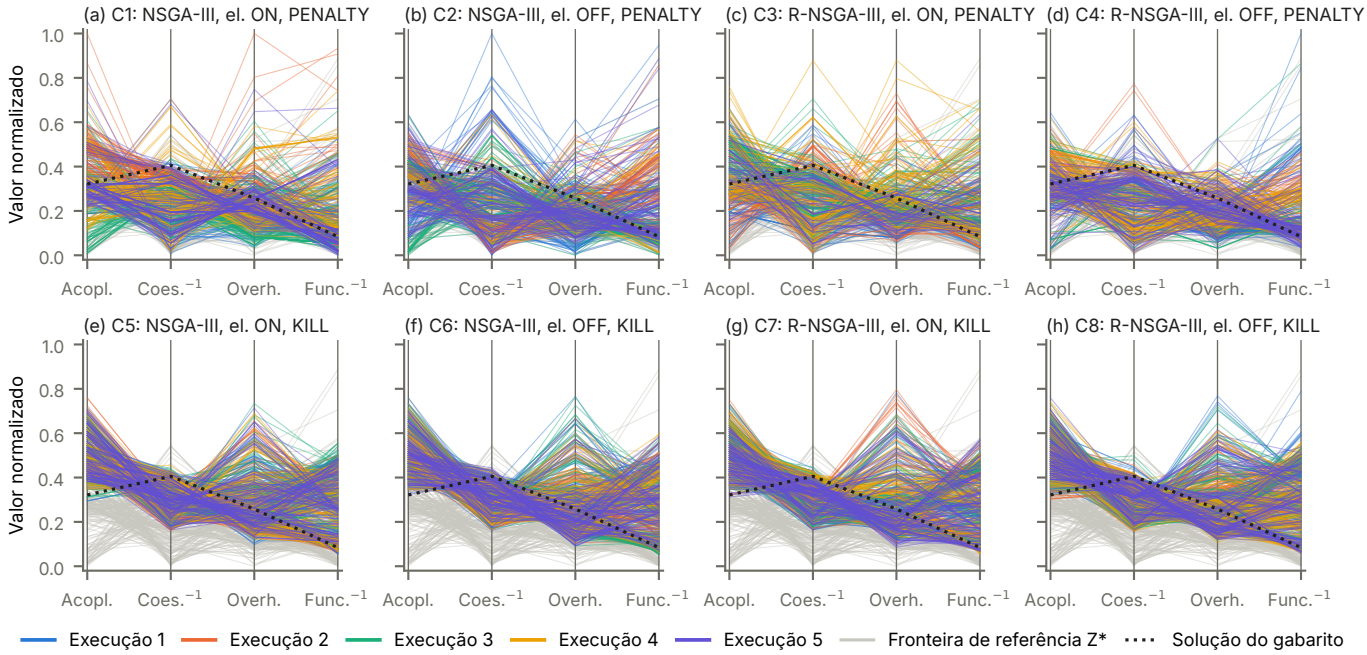


Figura 2. Fronteiras finais das oito configurações no spring-petclinic em coordenadas paralelas

referência expressa, portanto, o desalinhamento entre o que a ferramenta otimiza e o que o DM deseja. As fronteiras das configurações PENALTY (C1 a C4) cobrem boa parte da fronteira de referência, alcançando valores baixos em todos os objetivos, com grande diversidade entre as execuções; de fato, todos os 182 pontos da fronteira de referência provêm de execuções PENALTY, e nenhum de execuções KILL; na C2, melhor configuração segundo os indicadores, mesmo a execução de pior desempenho permanece próxima da fronteira de referência. As fronteiras das configurações KILL (C5 a C8) são mais concentradas, com as cinco execuções sobrepostas, e ficam afastadas da fronteira de referência, sobretudo no acoplamento e na coesão; contudo, passam próximas da linha que representa a solução do gabarito. Esse contraste antecipa o resultado central do estudo: a solução preferida pela persona tem acoplamento e *overhead* piores que as melhores soluções encontradas com PENALTY, o que é coerente com o perfil do Arquiteto Reflexivo, que aceita piorar esses objetivos em favor da coesão de domínio.

C. Indicadores de Qualidade

A Tabela IV apresenta a mediana de cada indicador nas cinco execuções de cada configuração, com o melhor valor em negrito e, na última linha, o p-valor do teste de Kruskal-Wallis; a Fig. 3 detalha o HV de cada execução no spring-petclinic. Para os dois sistemas, também foram gerados os boxplots de HV, IGD+ e ED por configuração, a estatística descritiva completa (média, mediana, desvio-padrão, quartis, intervalo interquartil, mínimo e máximo) e os histogramas e gráficos Q-Q utilizados na verificação da normalidade, por configuração e dos resíduos; esses resultados, com o *notebook* que os gera, estão disponíveis no material complementar, na pasta `dataset/analise_descritiva` (Seção III-F).

No spring-petclinic, a C2 (NSGA-III, elitismo inativo, PENALTY) obtém o melhor resultado nos três indicadores (HV 1,259; IGD+ 0,054; ED 0,531), seguida da C4 e da C1. As configurações formam dois blocos bem separados: com PENALTY (C1 a C4), as medianas variam entre 1,020 e 1,259 no HV, entre 0,054 e 0,121 no IGD+ e entre 0,531 e 0,588 na ED; com KILL (C5 a C8), ficam entre 0,713 e 0,733, entre 0,238 e 0,243 e entre 0,731 e 0,744, respectivamente, de modo que a pior configuração PENALTY supera a melhor KILL em todos os indicadores.

Os pressupostos da ANOVA não foram atendidos em dois dos três indicadores: o IGD+ violou a normalidade nas configurações C2 e C4 (Shapiro-Wilk, $p = 0{,}020$ e $p = 0{,}038$), e o HV violou a homogeneidade das variâncias (Levene, $p = 0{,}008$), uma vez que as execuções PENALTY variam muito mais que as KILL. Apenas na ED os pressupostos foram atendidos; nesse caso, a ANOVA seguida do teste de Tukey HSD, calculada como verificação e com os p-valores também ajustados por Benjamini-Hochberg, leva à mesma conclusão (16 pares significativos, todos PENALTY $\times$ KILL), e o Kruskal-Wallis foi mantido para uniformizar a análise. Os testes estatísticos confirmam essa separação. As diferenças entre as configurações são significativas nos três indicadores ($p < 0{,}001$) e, após a correção de Benjamini-Hochberg, todos os pares significativos do teste de Dunn opõem uma configuração PENALTY a uma KILL (13 pares no HV, 16 no IGD+ e 15 na ED; Tabela V). Agrupando as execuções por fator, PENALTY supera KILL em todas as comparações ($\hat{A}_{12} = 1{,}00$), enquanto o algoritmo ($\hat{A}_{12} = 0{,}48$ no HV) e o

elitismo ($\hat{A}_{12} = 0{,}41$) não apresentam efeito relevante.

Dentro de cada bloco, as diferenças não são significativas. Entre as configurações PENALTY, a C2 tem efeito grande sobre a C1 e a C3 no HV ($\hat{A}_{12} = 0{,}72$). A C3 foi a configuração PENALTY menos estável: duas execuções atingiram HV no nível da C2 (1,274 e 1,259), enquanto as outras três ficaram entre 0,942 e 1,020, o que explica sua mediana mais baixa, apesar dos valores altos visíveis na Fig. 3. Entre as configurações KILL, as medianas são praticamente iguais, e a ordem entre elas muda de um indicador para outro.

Esses resultados devem ser lidos à luz do que os indicadores medem. Uma configuração bem avaliada segundo eles é aquela cujas fronteiras alcançam bons valores de acoplamento, coesão, *overhead* e modularização, e não necessariamente aquela que preserva a decomposição desejada pela persona. Como se discute na Seção IV-D, as execuções que terminaram com a solução do gabarito foram todas KILL (C6, C7 e C8), justamente o bloco de pior desempenho nos indicadores, enquanto nenhuma execução da C2, melhor segundo eles, terminou com essa solução.

No jpetstore, as diferenças são menores. A C4 (R-NSGA-III, elitismo inativo, PENALTY) obtém as melhores medianas nos três indicadores, mas apenas o HV apresenta diferença significativa ($p = 0{,}013$), e somente os pares C4–C7 e C4–C8 resistem à correção. Ainda assim, a C4 apresenta efeito grande sobre todas as demais configurações no IGD+ ($\hat{A}_{12} \geq 0{,}76$). O tratamento de restrição volta a ser o fator de maior impacto: PENALTY supera KILL no HV ($\hat{A}_{12} = 0{,}85$) e no IGD+ ($\hat{A}_{12} = 0{,}72$), embora com efeito menor que no spring-petclinic.

D. Aderência às Preferências da Persona

A Fig. 4 apresenta a aderência ao gabarito na geração final das cinco execuções de cada configuração do spring-petclinic. Em cada execução, considerou-se a mais aderente das soluções extremas que a ferramenta registra a cada ponto de interação (a melhor solução em cada objetivo); a linha pontilhada indica 100%. A Tabela VI detalha a evolução da aderência média em cada ponto de interação. A aderência final variou entre 79,7% (C4) e 86,0% (C2), sem diferença significativa entre as configurações ($p = 0{,}93$). Em todas elas, a aderência partiu de cerca de 76% na geração 600 e aumentou até a geração final, em média sete pontos percentuais (de 3,6 na C4 a 9,9 na C2 e na C8), o que sugere que as intervenções do DM contribuíram para aproximar a população da decomposição desejada. Comparando apenas com o gabarito, portanto, não há uma configuração claramente superior: PENALTY e KILL terminaram com aderência média praticamente igual (83,1% e 83,4%).

A diferença entre as configurações aparece nos extremos e na forma como cada uma reage às intervenções. As quatro execuções que terminaram com uma solução 100% aderente ao gabarito são todas KILL (C6-E5, C7-E2, C8-E1 e C8-E3), e nenhuma execução PENALTY ultrapassou 97%. Com KILL, a aderência tende a permanecer estável entre as pausas e, quando atinge 100%, mantém esse valor até o fim (C7-E2 e C8-E1, desde a geração 950); com PENALTY, ela oscila, e valores altos alcançados após uma interação podem se perder na pausa seguinte (por exemplo, 85% na C1-E3 e 88% na C3-E1, que caem para 76% e 84% ao final). A análise conjunta da convergência (Fig. 1) e das fronteiras finais (Fig. 2) torna esse contraste mais claro. Como visto na Seção IV-A, com PENALTY o melhor acoplamento continua caindo após a geração 600, até 34 a 45 na geração final, enquanto com KILL ele permanece próximo ao da solução do gabarito (63). As fronteiras finais refletem esse comportamento: com PENALTY, em média 44% a 50% das soluções finais têm acoplamento menor que o da solução do gabarito; com KILL, essa proporção é de 0% a 1%, ou seja, as fronteiras não ultrapassam o gabarito no acoplamento e se concentram em torno da linha que o representa. Coerentemente, a solução de cada fronteira mais próxima do gabarito está, em média, um pouco mais perto dele com KILL (distância normalizada de 0,060 a 0,088) do que com PENALTY (0,081 a 0,114). Assim, embora a aderência média não distinga as configurações, o KILL preservou com mais frequência a decomposição indicada pelo DM, enquanto o PENALTY privilegiou os objetivos da ferramenta. De forma complementar, em oito das 20 execuções KILL a solução exibida em primeiro lugar ao final estava marcada pela ferramenta como modificada pelo DM, contra três das 20 execuções PENALTY; essa marca, contudo, não indica a aderência da solução, cuja composição não é exportada.

A posição da solução do gabarito no espaço de objetivos ajuda a explicar esse contraste. A solução 100% aderente recebeu valores de coesão distintos em execuções diferentes (1,019 e 0,950; Seção V) e, conforme o valor considerado, é dominada por 46 a 50 dos 182 pontos da fronteira de referência do spring-petclinic, isto é, há soluções que não são piores que ela em nenhum objetivo e são melhores em pelo menos um. Sua distância ao ideal (0,56 a 0,58) é semelhante à da solução mais próxima do ideal em cada fronteira KILL (0,52 a 0,61) e superior à das fronteiras PENALTY (0,24 a 0,55). Ou seja, segundo os valores de objetivo calculados pela própria ferramenta, a decomposição desejada pela persona não está entre as soluções ótimas. Assim, evoluir segundo os objetivos da ferramenta implica afastar-se da decomposição desejada: com PENALTY, a busca continua evoluindo, e a solução do DM tende a ser substituída por soluções melhores segundo esses objetivos; com KILL, os objetivos deixam de melhorar e a solução do DM tende a permanecer na população. Os dados não permitem distinguir se essa permanência decorre de uma resposta da busca à intervenção do DM ou de uma perda de diversidade da população; em ambos os casos, a preservação da preferência coincide com a ausência de melhoria segundo os objetivos. Coerentemente, os indicadores não distinguem as execuções KILL que preservaram essa solução das demais (HV médio de 0,729 contra 0,724; IGD+ de 0,241 contra 0,242).

Ao longo das execuções, também se observou que as soluções apresentadas pela ferramenta frequentemente distribuíam os métodos de uma mesma entidade de domínio entre diferentes microsserviços, incluindo métodos

de acesso ao mesmo atributo, como `Visit.getDate` e `Visit.setDate`. A Tabela III ilustra esse comportamento com a classe `Owner`, na solução de melhor coesão registrada pela ferramenta na geração 600, antes de qualquer interação do DM. Ou seja, mesmo a solução considerada mais coesa pelo objetivo da ferramenta divide a entidade. Em todas as configurações, os 11 métodos da classe foram distribuídos entre os quatro microsserviços; em todas, a escrita (`addPet`) e a leitura (`getPets`) da coleção de animais ficaram em microsserviços diferentes; e, em quatro delas, o mesmo ocorreu com `getTelephone` e `setTelephone`. Na tabela, que se refere à primeira execução de cada configuração, os pares *getter*/*setter* separados estão em negrito e a escrita e a leitura da coleção de animais, em itálico. O mesmo comportamento aparece na solução efetivamente exibida ao DM na primeira interação: na execução C8-E2, por exemplo, os quatro métodos da classe `Person` ficaram cada um em um microsserviço diferente (`getFirstName`, `setLastName`, `getLastName` e `setFirstName`, respectivamente no MS0, MS1, MS2 e MS3), e, nas nove execuções em que essa solução foi registrada, as classes `Owner`, `Pet` e `NamedEntity` apareceram divididas em todas.

Essa distribuição é relevante para a arquitetura resultante. Um dos princípios dos microsserviços é a coesão de domínio: cada serviço deve modularizar uma única funcionalidade de negócio [4], e padrões de migração recomendam identificar os serviços a partir dos subdomínios do negócio ou da posse dos dados, agrupando cada conjunto coeso de entidades com a lógica que as manipula [11]. Quando os métodos de uma mesma entidade ficam em microsserviços distintos, surgem dificuldades práticas. Na persistência, a entidade deixa de ter um único serviço responsável por armazená-la, o que tende a exigir um banco de dados compartilhado ou a sincronização dos dados entre serviços, situação apontada como uma das limitações dos sistemas legados que motivam a migração [7]. Na manutenção, alterações na entidade, como a inclusão de um atributo, passam a exigir mudanças coordenadas em mais de um serviço, o que pode comprometer a implantação independente.

Essa observação suscita questões que, na experiência deste estudo, merecem investigação. A primeira é se a fragmentação tende a se reforçar ao longo da busca: uma entidade já dividida possivelmente permanece dividida nas gerações seguintes, caso as soluções descendentes herdem a partição das soluções que lhes deram origem. A segunda refere-se aos atributos da entidade: se `getFirstName` e `setFirstName` estão em serviços distintos, não está claro se o atributo `firstName` seria replicado em ambos, o que exigiria garantir a consistência entre as cópias e coordenar alterações de tipo ou de validação, ou se o acesso a ele passaria a exigir chamadas entre serviços. Nesse segundo caso, cabe verificar se esse custo de comunicação está contemplado no objetivo de *overhead* de rede, que parece considerar as chamadas entre métodos, mas não necessariamente o acesso compartilhado a um mesmo dado. A terceira refere-se ao banco de dados: não fica definido qual serviço seria responsável pela tabela de proprietários quando os métodos que a leem e a escrevem estão em quatro serviços diferentes. Essas questões reforçam a sugestão, discutida na Seção V, de considerar a posse dos dados e a coesão por entidade na formulação do problema.

Uma possibilidade é que os objetivos, ao privilegiar as relações de chamada entre métodos, sejam pouco sensíveis à separação de métodos que manipulam o mesmo dado; nesse caso, essa separação poderia também favorecer a divisão de operações que dependem desses métodos. Esse aspecto pode contribuir para compreender por que a solução do gabarito, que mantém as classes inteiras, é dominada por soluções que as dividem, e por que parte das correções do DM foi desfeita entre uma interação e outra, constituindo um ponto interessante para investigações futuras.

E. Respostas às Questões de Pesquisa

RQ1. Segundo os indicadores de qualidade, a melhor configuração no spring-petclinic foi a C2 (NSGA-III, elitismo inativo, PENALTY), que obteve as melhores medianas nos três indicadores; no jpetstore, foi a C4 (R-NSGA-III, elitismo inativo, PENALTY). O fator determinante foi o tratamento de restrição: PENALTY superou KILL em todos os indicadores no spring-petclinic ($\hat{A}_{12} = 1{,}00$); no jpetstore, a vantagem do PENALTY aparece no tamanho de efeito ($\hat{A}_{12} = 0{,}85$ no HV), mas a diferença foi significativa apenas no HV, e somente entre a C4 e as configurações C7 e C8. O algoritmo e o elitismo não produziram diferenças significativas. Assim, mais do que uma configuração isolada, o desempenho segundo os indicadores depende principalmente do tratamento de restrição.

RQ2. Parcialmente. No spring-petclinic, a aderência à decomposição desejada pela persona aumentou ao longo das interações do DM em todas as configurações (de cerca de 76% para 83% em média), e, comparando apenas com o gabarito, não há uma configuração claramente superior. Contudo, a convergência e as fronteiras finais mostram que o KILL preservou melhor a intervenção do DM: seus objetivos deixam de melhorar a partir da primeira interação, suas fronteiras permanecem próximas da solução desejada, e apenas ele manteve essa solução até o fim (quatro execuções com 100% de aderência). Esse comportamento, porém, pode refletir tanto uma resposta da busca às soluções do DM quanto a perda de diversidade da população. O PENALTY, ao contrário, continua otimizando os objetivos e se afasta dela. Assim, os indicadores de qualidade e a preferência do DM apontaram em direções opostas: a configuração mais bem avaliada pelos indicadores não é a que melhor preserva a intenção do DM. Observou-se ainda que as soluções apresentadas tendem a distribuir as entidades de domínio entre diferentes microsserviços, o que sugere que preferências por fronteiras baseadas no domínio, como a da persona adotada, possam se beneficiar de critérios complementares aos objetivos atuais.

V. Sugestões de Melhorias para a Ferramenta

A partir dos resultados e da experiência de uso ao longo das execuções, identificaram-se oportunidades de evolução

Tabela III
DISTRIBUIÇÃO DOS MÉTODOS DA CLASSE OWNER NA SOLUÇÃO DE MELHOR COESÃO DA GERAÇÃO 600

Config.	MS0	MS1	MS2	MS3
C1	addVisit	getAddress, getCity, getPet, getTelephone, setAddress, setCity, setTelephone	*addPet*	*getPets*, toString
C2	addVisit, getTelephone, setTelephone	getAddress, getCity, setAddress, setCity	*addPet*, getPet	*getPets*, toString
C3	addVisit, getPet	getAddress, getCity, setAddress, setCity, **setTelephone**	*addPet*	*getPets*, **getTelephone**, toString
C4	*getPets*	getAddress, getCity, setAddress, setCity, **setTelephone**	getPet, **getTelephone**	*addPet*, addVisit, toString
C5	addVisit, getPet	getAddress, getCity, getTelephone, setAddress, setCity, setTelephone	*addPet*	*getPets*, toString
C6	*getPets*, **getTelephone**, toString	getAddress, getCity, setAddress, setCity, **setTelephone**	*addPet*	addVisit, getPet
C7	*getPets*, **getTelephone**, toString	getAddress, getCity, setAddress, setCity, **setTelephone**	*addPet*, getPet	addVisit
C8	*getPets*, toString	getAddress, getCity, getTelephone, setAddress, setCity, setTelephone	*addPet*	addVisit, getPet

Tabela IV
MEDIANA DOS INDICADORES DE QUALIDADE POR CONFIGURAÇÃO (CINCO EXECUÇÕES, GERAÇÃO FINAL)

				spring-petclinic			jpetstore		
Config.	Algoritmo	Elitismo	Restrição	HV↑	IGD+↓	ED↓	HV↑	IGD+↓	ED↓
C1	NSGA-III	ativo	PENALTY	1,118	0,089	0,535	0,995	0,127	0,643
C2	NSGA-III	inativo	PENALTY	**1,259**	**0,054**	**0,531**	0,925	0,149	0,677
C3	R-NSGA-III	ativo	PENALTY	1,020	0,121	0,588	0,945	0,153	0,634
C4	R-NSGA-III	inativo	PENALTY	1,179	0,076	0,532	**1,055**	**0,088**	**0,555**
C5	NSGA-III	ativo	KILL	0,713	0,239	0,740	0,830	0,167	0,656
C6	NSGA-III	inativo	KILL	0,733	0,243	0,731	0,867	0,149	0,645
C7	R-NSGA-III	ativo	KILL	0,718	0,238	0,737	0,805	0,185	0,647
C8	R-NSGA-III	inativo	KILL	0,728	0,240	0,744	0,768	0,187	0,562
			p (Kruskal-Wallis)	<0,001	<0,001	<0,001	0,013	0,130	0,166

↑: maior é melhor; ↓: menor é melhor. ED: média das distâncias euclidianas das soluções da fronteira ao ponto ideal.

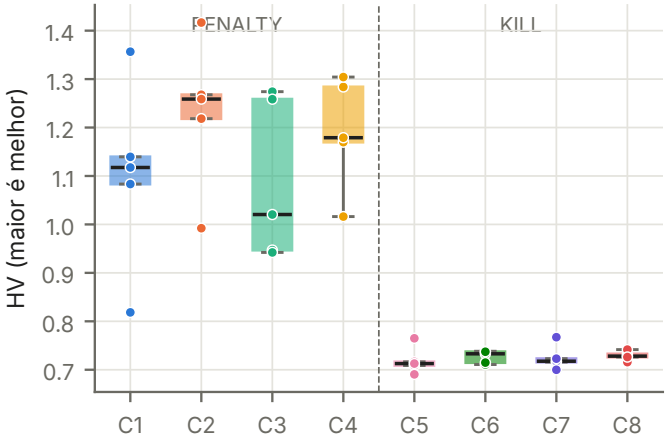


Figura 3. Hypervolume por configuração no spring-petclinic

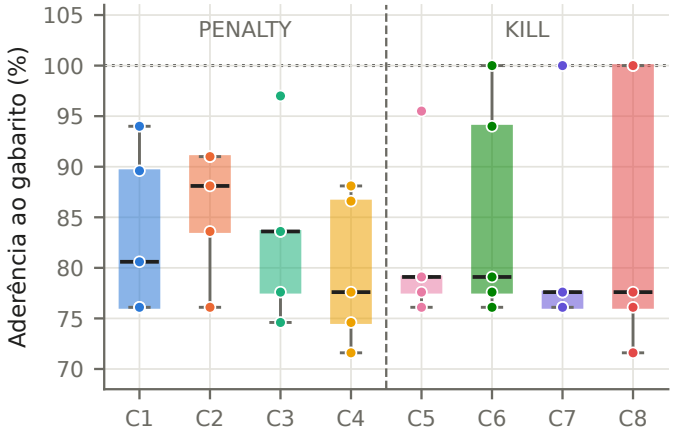


Figura 4. Aderência ao gabarito por configuração no spring-petclinic

da i-toMicroservices, relacionadas à busca, à interface e à robustez da ferramenta.

Quanto à busca, um critério opcional de coesão de domínio, na forma de um objetivo ou de uma restrição ativável pelo DM, poderia favorecer que os métodos de uma mesma entidade, ou que acessam o mesmo atributo, permaneçam juntos, atendendo a perfis de DM que privilegiam fronteiras baseadas no domínio, como o da persona adotada (Tabela III). Na mesma direção, considerar aspectos de banco de dados na busca, como as relações de acesso entre métodos e tabelas e o compartilhamento de tabelas entre microsserviços, poderia apoiar a posse dos dados por cada serviço. Essa possibilidade é coerente com a ausência de uma função objetivo para avaliar o impacto no banco de dados, apontada como trabalho futuro na análise da abordagem [11], com o uso de uma métrica de pureza transacional, baseada no compartilhamento de tabelas,

Tabela V
TESTE DE DUNN: p-VALORES DAS 28 COMPARAÇÕES PAR A PAR, AJUSTADOS POR BENJAMINI-HOCHBERG

	spring-petclinic			jpetstore		spring-petclinic			jpetstore
Par	**HV**	**IGD+**	**ED**	**HV**	**Par**	**HV**	**IGD+**	**ED**	**HV**
C1 × C2	0,760	0,776	1,000	0,613	C3 × C5	**0,019**	**0,028**	**0,035**	0,448
C1 × C3	0,914	0,970	0,861	0,598	C3 × C6	0,053	**0,028**	0,085	0,598
C1 × C4	0,822	0,961	0,992	0,665	C3 × C7	**0,037**	**0,043**	**0,030**	0,287
C1 × C5	**0,018**	**0,028**	**0,013**	0,204	C3 × C8	0,070	**0,035**	**0,030**	0,231
C1 × C6	**0,043**	**0,028**	**0,030**	0,287	C4 × C5	**0,011**	**0,023**	**0,014**	0,083
C1 × C7	**0,030**	**0,037**	**0,013**	0,104	C4 × C6	**0,019**	**0,023**	**0,030**	0,152
C1 × C8	0,057	**0,030**	**0,013**	0,083	C4 × C7	**0,018**	**0,028**	**0,013**	**0,049**
C2 × C3	0,760	0,739	0,861	0,914	C4 × C8	**0,024**	**0,028**	**0,013**	**0,043**
C2 × C4	0,914	0,951	0,992	0,398	C5 × C6	0,760	0,978	0,897	0,759
C2 × C5	**0,011**	**0,015**	**0,013**	0,398	C5 × C7	0,846	0,961	0,992	0,759
C2 × C6	**0,018**	**0,015**	**0,030**	0,598	C5 × C8	0,760	0,961	0,992	0,628
C2 × C7	**0,018**	**0,019**	**0,013**	0,277	C6 × C7	0,914	0,961	0,861	0,598
C2 × C8	**0,020**	**0,016**	**0,013**	0,204	C6 × C8	0,914	0,961	0,861	0,501
C3 × C4	0,760	0,961	0,861	0,361	C7 × C8	0,846	0,961	0,992	0,838

Em negrito, $p \leq 0{,}05$. C1–C4: PENALTY; C5–C8: KILL.
Pares significativos no spring-petclinic: 13 (HV), 16 (IGD+) e 15 (ED), todos PENALTY × KILL.
No jpetstore, o Dunn só se aplica ao HV, único indicador com Kruskal-Wallis significativo ($p = 0{,}013$).

Tabela VI
ADERÊNCIA MÉDIA AO GABARITO (%) EM CADA PONTO DE INTERAÇÃO NO SPRING-PETCLINIC

Config.	**Restr.**	**GER 600**	**GER 820**	**GER 950**	**Final**	**Exec. 100%**
C1	PENALTY	75,8	80,9	83,6	83,3	0
C2	PENALTY	76,1	77,9	80,6	**86,0**	0
C3	PENALTY	76,4	80,6	82,1	83,3	0
C4	PENALTY	76,1	78,5	77,9	79,7	0
C5	KILL	77,6	77,6	77,6	81,5	0
C6	KILL	76,1	76,1	77,3	85,4	1
C7	KILL	76,4	76,4	81,5	81,5	1
C8	KILL	75,2	77,9	83,3	85,1	2

GER 600: antes da primeira interação; Final: geração 999. Média das cinco execuções.

na avaliação da i-toMicroservices [3], e com outras técnicas de decomposição que já exploram as relações entre métodos e tabelas [2].

Ainda quanto à busca, seria útil manter a solução modificada pelo DM identificada e acessível na lista final de candidatas, independentemente de sua posição, uma vez que soluções 100% aderentes nem sempre permaneceram em destaque até o fim da busca. Nas configurações com KILL, em que a busca estagnou a partir da primeira interação (Fig. 1), mecanismos que recuperem a diversidade da população após as intervenções do DM merecem investigação. Observou-se também, no spring-petclinic, que uma mesma partição recebeu valores de coesão distintos em execuções diferentes (1,019 e 0,950). Como o DM compara as soluções por esses valores, documentar os fatores que influenciam o cálculo, ou sinalizar quando um valor pode variar entre execuções, facilitaria a interpretação dos resultados.

A usabilidade também se mostrou um desafio. A cada interação, o DM lida com grande volume de informação (soluções candidatas, valores de cinco objetivos e composição de cada microsserviço) e precisa tomar muitas decisões e executar muitas ações, o que tornou o uso exigente e a ferramenta difícil de compreender, sobretudo nas primeiras execuções. Parte dessa dificuldade é inerente à complexidade do problema: exibir menos informação pode ocultar elementos relevantes para a decisão, enquanto exibir tudo sobrecarrega o DM. Equilibrar esses dois extremos, por exemplo com níveis de detalhe ajustáveis ao perfil do DM, é uma oportunidade de evolução. Nesse contexto, um objetivo central da interface é evitar que o DM cometa erros: como suas decisões orientam a busca, um engano na operação se converte em um sinal incorreto para o algoritmo e pode conduzir a resultados indesejados.

Alguns ajustes pontuais poderiam contribuir nesse sentido. Cada microsserviço é apresentado como uma lista de métodos identificados pelo nome completo, incluindo o pacote, que pode chegar a dezenas de itens, o que dificulta sua leitura. Apresentar cada microsserviço como uma lista expansível de classes, em que um controle (por exemplo, "+") exibe os métodos de cada uma com nomes curtos e o nome completo sob demanda, reduziria o volume de informação exibido de uma só vez e permitiria identificar rapidamente quais classes compõem o microsserviço e se a composição faz sentido. Indicar, para cada classe, quantos de seus métodos estão no microsserviço (por exemplo, `Owner` 3/11) tornaria visível, sem conferência manual, quando uma classe está dividida, cabendo ao DM decidir se mantém ou corrige essa divisão. O mesmo agrupamento permitiria mover uma classe inteira quando apropriado, o que reduziria o número de operações e, com ele, a chance de erro, e permitiria ao DM testar rapidamente o efeito de uma reorganização sobre os objetivos, sem prejuízo da movimentação individual de métodos. Na operação de mover, os métodos perdem a cor de seleção ao chegar ao microsserviço de destino, de modo que, em listas longas, não é trivial conferir se a operação foi realizada; mantê-los destacados facilitaria essa verificação. A existência de dois botões de mover, um para o primeiro método e outro para os métodos selecionados, também pode gerar confusão, e diferenciá-los com mais clareza, ou unificá-los, reduziria esse risco. Além disso, a cada novo par de microsserviços, o DM precisa reiniciar a sequência de seleção da operação, dos

microsserviços e dos métodos; permitir trocar o par na mesma tela tornaria o processo mais ágil. Por fim, os nomes atribuídos pelo DM aos microsserviços nem sempre se mantiveram entre as interações, o que exigiu renomeá-los e aumentou o tempo de decisão.

Quanto à robustez, em algumas interações a operação de mover foi registrada sem métodos, sem indicação ao DM, o que só foi percebido por conferência manual dos tamanhos dos microsserviços; um aviso nesses casos e a possibilidade de desfazer operações reduziriam o retrabalho. Exportar automaticamente os dados a cada interação permitiria retomar uma execução interrompida por falha sem perder o trabalho já realizado, e incluir nos pacotes exportados a composição da solução exibida ao DM, e não apenas seus valores de objetivo, permitiria medir diretamente sua aderência às preferências do DM.

Outro aspecto observado refere-se à preparação do ambiente. Para utilizar a ferramenta, cada participante precisou instalá-la e configurá-la localmente, o que demanda tempo antes mesmo do início das execuções e expõe a avaliação a problemas específicos de cada máquina. O uso de contêineres Docker reduz os riscos associados a versões de dependências, mas não elimina por completo eventuais dificuldades de instalação. Disponibilizar a ferramenta em um servidor, acessível pelo navegador, permitiria que o DM se concentrasse nas decisões de modularização, sem preocupações com instalação, versões ou falhas do ambiente local, e poderia favorecer a participação de um número maior de avaliadores.

As falhas têm impacto particular em avaliações como a deste estudo. Cada execução exige atenção contínua do DM, registro das decisões e conferência das operações, em um processo longo e cansativo. Quando uma falha ocorre nas etapas finais e a execução precisa ser refeita, perde-se não apenas o tempo de processamento, mas também as anotações, a análise e o empenho dedicados àquela sessão, o que pode desmotivar justamente os usuários mais engajados em contribuir com a avaliação. Ocorrências desse tipo são esperadas em ferramentas de pesquisa em desenvolvimento, e reduzi-las depende de ciclos repetidos de uso e ajuste. Esses ciclos, porém, são difíceis de obter em projetos de mestrado e doutorado, já que cada execução é longa e complexa e os usuários especializados capazes de avaliá-la dispõem de pouco tempo. Além disso, o esforço de aprimorar a usabilidade e a robustez costuma ter menos reconhecimento como contribuição científica do que a proposição de novas técnicas, embora seja decisivo para que ferramentas de pesquisa possam ser avaliadas e adotadas. Avaliações conduzidas em colaboração com os desenvolvedores da ferramenta, como a deste estudo, são uma forma de alimentar esse ciclo.

VI. Considerações Finais

Este trabalho avaliou se três mecanismos recentemente incorporados à i-toMicroservices, o elitismo das soluções do DM, o tratamento da restrição de balanceamento e o algoritmo R-NSGA-III, são benéficos no uso interativo da ferramenta. Para isso, oito configurações foram executadas em dois sistemas, com um DM humano guiado por uma persona com preferência por fronteiras baseadas no domínio, totalizando 80 execuções.

Em relação à RQ1, o tratamento de restrição foi o fator determinante: PENALTY superou KILL nos indicadores de qualidade, de forma ampla no spring-petclinic; no jpetstore, a diferença foi significativa apenas no HV, e somente entre a C4 e as configurações C7 e C8, com a C2 e a C4 como melhores configurações, enquanto o algoritmo e o elitismo não produziram diferenças significativas. Em relação à RQ2, os resultados no spring-petclinic estão parcialmente condizentes com as preferências da persona: a aderência à decomposição desejada aumentou ao longo das intervenções do DM em todas as configurações, sem diferença clara de aderência média entre elas; contudo, a convergência e as fronteiras finais indicam que o KILL preservou melhor a decomposição indicada pelo DM, sendo a única restrição sob a qual ela foi mantida até o fim, embora isso possa refletir também a perda de diversidade da população. Os indicadores de qualidade e a aderência à preferência do DM apontaram, assim, em direções opostas, e a solução do gabarito mostrou-se dominada segundo os próprios objetivos da ferramenta. Esse resultado sugere que, na avaliação de abordagens interativas, os indicadores tradicionais podem não refletir o quanto as soluções atendem às preferências do DM, e que medidas de aderência a essas preferências são um complemento importante. De modo geral, os resultados reforçam o potencial da otimização interativa na identificação de microsserviços: no spring-petclinic, a aderência à arquitetura desejada aumentou ao longo das interações do DM em todas as configurações avaliadas, e as oportunidades de evolução identificadas podem contribuir para que essa participação se reflita de forma ainda mais consistente nas soluções finais.

Como trabalhos futuros, sugere-se replicar o experimento com outras personas, mais tomadores de decisão e sistemas maiores; investigar critérios de coesão de domínio e de impacto no banco de dados, em linha com as direções já apontadas pelos autores da abordagem; medir a aderência diretamente na solução exibida ao DM; e incorporar medidas de aderência às preferências do DM na avaliação de abordagens interativas.

Referências

[1] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, "Decomposition of monolith applications into microservices architectures: A systematic review," *IEEE Trans. Softw. Eng.*, vol. 49, no. 8, pp. 4213–4242, 2023, doi: 10.1109/TSE.2023.3287297.

[2] Y. Wang, S. Bornais, and J. Rubin, "Microservice decomposition techniques: An independent tool comparison," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2024, pp. 1295–1307, doi: 10.1145/3691620.3695504.

[3] J. Castanho, C. G. Britez, W. M. Freire, A. M. M. M. Amaral, T. E. Colanzi, and W. K. G. Assunção, "Human in the loop for microservice identification: An exploratory study," in *Proc. Brazilian Symp. Softw. Components, Architectures, and Reuse (SBCARS)*, São Paulo, Brazil, 2026.

[4] L. Carvalho, A. Garcia, T. E. Colanzi, W. K. G. Assunção, J. A. Pereira, B. Fonseca, M. Ribeiro, M. J. de Lima, and C. Lucena, "On the performance and adoption of search-based microservice identification with

toMicroservices," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, 2020, pp. 569–580, doi: 10.1109/ICSME46990.2020.00060.

[5] W. K. G. Assunção, T. E. Colanzi, L. Carvalho, J. A. Pereira, A. Garcia, M. J. de Lima, and C. Lucena, "A multi-criteria strategy for redesigning legacy features as microservices: An industrial case study," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, 2021, pp. 377–387, doi: 10.1109/SANER50967.2021.00042.

[6] J. Castanho, J. Gomes, T. E. Colanzi, A. M. M. M. Amaral, W. K. G. Assunção, and A. Garcia, "i-toMicroservices: Empowering architects in the interactive modernization of monolithic systems," in *Proc. Brazilian Symp. Softw. Eng. (SBES)*, São Paulo, Brazil, 2026.

[7] L. Carvalho, T. E. Colanzi, W. K. G. Assunção, A. Garcia, J. A. Pereira, M. Kalinowski, R. de Mello, M. J. de Lima, and C. Lucena, "On the usefulness of automatically generated microservice architectures," *IEEE Trans. Softw. Eng.*, vol. 50, no. 3, pp. 651–667, Mar. 2024, doi: 10.1109/TSE.2024.3361209.

[8] M. Li, T. Chen, and X. Yao, "How to evaluate solutions in Pareto-based search-based software engineering: A critical review and methodological guidance," *IEEE Trans. Softw. Eng.*, vol. 48, no. 5, pp. 1771–1799, 2022.

[9] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints," *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 577–601, 2014, doi: 10.1109/TEVC.2013.2281535.

[10] Y. Vesikar, K. Deb, and J. Blank, "Reference point based NSGA-III for preferred solutions," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, 2018, pp. 1587–1594.

[11] W. K. G. Assunção, T. E. Colanzi, L. Carvalho, A. Garcia, J. A. Pereira, M. J. de Lima, and C. Lucena, "Analysis of a many-objective optimization approach for identifying microservices from legacy systems," *Empirical Softw. Eng.*, vol. 27, no. 2, art. 51, 2022, doi: 10.1007/s10664-021-10049-7.

[12] A. Arcuri and L. Briand, "A Hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Softw. Test., Verif. Reliab.*, vol. 24, no. 3, pp. 219–250, 2014, doi: 10.1002/stvr.1486.